

MEDIATEK

CUXV5-26330/27342

Issue Overview

- 问题概述
- 问题分析
- 问题原因
- 解决方案

db.00.KernelAPI 生成机制

- 接口:

- MTK_AEE 模块提供的对外的接口，以便于各个模块调用主动生成DB，并且从DB中获取更全的系统咨询。
 - aee_kernel_exception_api ()
 - aee_kernel_warning_api ()

- 影响:

- 主动trigger 一个db.00.KernelAPI
- aee_kernel_exception_api () 和 aee_kernel_warning_api () 接口调用不会造成系统重启

问题分析

```
Backtrace: [<fffffd124a1dd1c>] cache_parity_irq_work+0x400/0x4e4 [cache_parity] [<fffffd129ada1f4>] process_one_work+0x204/0x598 [<fffffd129ada804>] worker_thread+0x220/0x42c
[<fffffd129ae19cc>] kthread+0x100/0x1d0 [<fffffd129a1718c>] ret_from_fork+0x10/0x20
[ 380.765471] irq/529-0.nebul: vhost-mtk: [name:hyp_power_service&]handle_clk_request: invalid server vmid=-1
[ 380.766016] irq/529-0.nebul: vhost-mtk: [name:hyp_power_service&]handle_clk_request: invalid server vmid=-1
[ 380.766221] irq/529-0.nebul: vhost-mtk: [name:hyp_power_service&]handle_clk_request: invalid server vmid=-1
```

src/kernel/modules/mt8676/virt/grt/power_service/clk.c

```
140 int handle_clk_request(struct power_service *server, struct mtk_request *req,
141                        struct mtk_response *resp)
142 {
143     struct clk_req *clk_req = &req->clk_req;
144     struct clk_service *clk_service = &(server->clk_service);
145     struct clk *clk, *parent_clk;
146     int ret = 0;
147     const char *clk_name = clk_req->name;
148     struct clk_map_entry *entry;
149     int vmid = server->vmid;
150
151     if(vmid >= UOS COUNT || vmid < 0) {
152         pr_err("%s: invalid server vmid=%d\n", __func__, vmid);
153         return -EINVAL;
154     }
155     clk = get_clk_by_name(clk_service, clk_req->name, &entry);
156     if (clk == NULL) {
157         pr_err("get_clk_by_name can't find clk %s\n", clk_req->name);
158         return 0;
159     }
```

```
[ 380.783315] swapper/3: [name:cache_parity&]ecc_dump_debug_info is not implemented
[ 380.783335] swapper/3: [name:cache_parity&]Cache ECC error, cpu3 serviced irq28
[ 380.783348] swapper/3: [name:cache_parity&]SERR[7:0] = 0xc, bypass this error!
[ 380.783445] swapper/3: [name:cache_parity&]ecc_dump_debug_info is not implemented
[ 380.783458] swapper/3: [name:cache_parity&]Cache ECC error, cpu3 serviced irq28
[ 380.783466] swapper/3: [name:cache_parity&]SERR[7:0] = 0xc, bypass this error!
[ 380.783472] swapper/3: [name:cache_parity&]cache_parity_isr_v3 disable irq 28 due to trigger over than 1 times.
[ 380.859350] swapper/0: [name:cache_parity&]ecc_dump_debug_info is not implemented
[ 380.859378] swapper/0: [name:cache_parity&]Cache ECC error, cpu0 serviced irq26(expected cpu1)
[ 380.859412] swapper/0: [name:cache_parity&]Cache ECC error, irq 26, misc0_el1: 0x0000000000000000, status_el1: 0x0000000000000000
[ 380.859426] swapper/0: [name:cache_parity&]cache_parity_isr_v3 disable irq 26 due to trigger over than 1 times.
```

结论:

vmid = -1 会导致对应的 clk request无法继续往下设定

- log File: SYS_KERNEL_LOG

[702.538578] irq/529-0.nebul: vhost-mtk: [name:hyp_power_service&]handle_clk_request: clk name=**mmp1l**, vmid=-1, ops=2
 [702.538590] irq/529-0.nebul: vhost-mtk: [name:hyp_power_service&]handle_clk_request: invalid server **vmid=-1**

venc_sel	CLK_CFG_12[27:24]	27	24	31	6	mmp1l_d6	20	VCORE
					7	mainpll_d5_d2	20	VCORE
					8	mainpll_d3	20	VCORE
					9	mmp1l_d9	20	VCORE
					10	mmp1l_d4	20	VCORE

结论1:

- 因为 vmid = -1 ,所以 clk: mmp1l 无法设定成功。

- log File: SYS_LAST_CPU_BUS

write entry = 63, valid = 0x1, non-secure = 0x1, write master id = 0x0, slave address = **0x17000008**, data_size = 0x2, burst_length = 0x0

write entry = 63, valid = 0x1, non-secure = 0x1, write master id = 0x0, slave address = **0x17040090**, data_size = 0x2, burst_length = 0x0

VENC1_APB_S	VENC	0x1700_0000	0x1700_FFFF	D	64KB	64KB	venc_global_con
VENC1_APB_S-1		0x1701_0000	0x1701_FFFF	D	64KB	4KB	smi_larb7
VENC1_APB_S-2		0x1702_0000	0x1702_FFFF	D	64KB	64KB	venc
VENC1_APB_S-3		0x1703_0000	0x1703_FFFF	D	64KB	4KB	jpgenc
VENC1_APB_S-4		0x1704_0000	0x1704_FFFF	D	64KB	4KB	jpgdec

结论2:

kernel API Dump的产生是因为bus_tracker抓到transaction返回了slave error。 **0x17000008** 属于 venc, **0x17040090** 属于 jpg

Review code : arch / arm64 / boot / dts / mediatek / mt6897.dts

- VENC/jpeg encode/jpeg decode 使用 mmp1l_d4 档位
- mmp1l 是一个pll, 很多clock都会用到, 例如: venc
- venc domain 其他模块也会用到, 例如: video encode/jpeg encode/ jpeg decode

结论3:

mmp1l 没有设成功, VENC/jpeg encode/jpeg decode 都有可能被bus_tracker抓到transaction返回了slave error

问题分析-DB生成流程, 以及ecc error 打印说明

src: /src/kernel/linux/v6.1_mt8676/co_device_module/drivers/misc/mediatek/sda/cache-parity.c

```

297 static irqreturn_t cache_parity_isr_v3(int irq, void *dev_id)
298 {
299     u32 hwirq, serr;
300     int i, idx, cpu;
301     u64 misc0, status;
302     u64 sel = 0x1;
303     static const struct midr_range cpu_list[] = {
304         MIDR_ALL_VERSIONS(MIDR_CORTEX_A510), /* KLEIN */
305     };
306 };
307
308 ecc_dump_debug_info();
309
310 cpu = raw_smp_processor_id();
311 if ((cache_parity.record[idx].v3.cpu != nr_cpu_ids) &&
312     (cpu != cache_parity.record[idx].v3.cpu))
313     ECC_LOG("Cache ECC error, cpu%d serviced irq%d(%s%d)\n",
314         cpu, irq, "expected cpu",
315         cache_parity.record[idx].v3.cpu);
316
317 ECC_LOG("Cache ECC error, cpu%d serviced irq%d\n", cpu, irq);
318 ECC_LOG("SERR[7:0] = 0x%x, bypass this error!\n", serr);
319 goto check_nr_err;
320
321 }
322
323 schedule_work(&cache_parity.work);
324
325 }
326
327 ECC_LOG("Cache ECC error, %s %d, %s: 0x%016llx, %s: 0x%016llx\n",
328     "irq", irq, "misc0_el1", misc0, "status_el1", status);
329
330

```

```

126 static void cache_parity_irq_work(struct work_struct *w)
127 {
128     static char *buf;
129     int n, i;
130     u64 status;
131
132     if (!buf) {
133         buf = kmalloc(PAGE_SIZE, GFP_KERNEL);
134         if (!buf)
135             goto call;
136     }
137     n = 0;
138
139     * KERNEL_REPORT_LENGTH (should be 344), if oversize, it will be cut off.
140
141     aee_kernel_exception("cache parity", "%s\n%s", buf, "CRDISPATCH_KEY:Cache Parity Issue");
142
143     pr_debug("CRDISPATCH_KEY:Cache Parity Issue");
144     if (!buf)
145         kfree(buf);
146
147

```

```

[ 380.783315] swapper/3: [name:cache_parity]&ecc_dump_debug_info is not implemented
[ 380.783335] swapper/3: [name:cache_parity]&Cache ECC error, cpu3 serviced irq28
[ 380.783348] swapper/3: [name:cache_parity]&SERR[7:0] = 0xc, bypass this error!
[ 380.783445] swapper/3: [name:cache_parity]&ecc_dump_debug_info is not implemented
[ 380.783458] swapper/3: [name:cache_parity]&Cache ECC error, cpu3 serviced irq28
[ 380.783466] swapper/3: [name:cache_parity]&SERR[7:0] = 0xc, bypass this error!
[ 380.783472] swapper/3: [name:cache_parity]&cache_parity_isr_v3 disable irq 28 due to trigger over than 1 ti
[ 380.859350] swapper/0: [name:cache_parity]&ecc_dump_debug_info is not implemented
[ 380.859378] swapper/0: [name:cache_parity]&Cache ECC error, cpu0 serviced irq26(expected cpu1)
[ 380.859412] swapper/0: [name:cache_parity]&Cache ECC error, irq 26, misc0_el1: 0x0000000000000000, sta
[ 380.859426] swapper/0: [name:cache_parity]&cache_parity_isr_v3 disable irq 26 due to trigger over than 1 ti

```

- bus_tracker抓到transaction返回了slave error, 触发cache_parity_isr_v3 中断响应。
- 看到的"Ecc error" 只是打印, 不是真的出现cache error, 只是slave error出现后, 反应到cache ecc 这边。

问题原因 (vmid=-1) -详情见谦川解析

• vmid =-1发生原因

- 公版修改了vm 关机流程，改为先跑 acrn_vm_destroy(), 再跑 acrn_dev_release()。
- 因为 acrn_vm_destroy() 会将 vm->vmid 改为非法值，所以增加了vm->saved_vmid。
- 在acrn_vm_destroy() 函数末尾初始化，供 acrn_dev_release() 使用。
- 遗漏了先跑 acrn_dev_release() 情况，导致vm->saved_vmid 没有初始化，最终导致此问题。

```

128  *   vm->monitor_page = NULL;
129  *   }
130
131  *   vm->is_destroying = true;
132  *   wake_up_interruptible(&vm->async_ioreqs_wq);
133  *   acrn_async_ioreq_deinit(vm);
134  *   acrn_vm_all_ram_unmap(vm);
135  *   acrn_vm_destroy(vm);
136
137  *   dev_dbg(acrn_dev.this_device, "VM %u destroyed.\n", vm->vmid);
138  *   vm->saved_vmid = vm->vmid;
139  *   vm->vmid = ACRN_INVALID_VMID;
140  *   return 0;
141  *   }

```

acrn_vm_destroy() 函数

```

570 static int acrn_dev_release(struct inode *inode, struct file *filp)
571 {
572     struct acrn_vm *vm = filp->private_data;
573     int vmid = vm->saved_vmid;
574
575     acrn_vm_destroy(vm);
576     kfree(vm);
577
578     // GRT:TODO add power service later.
579     guest_vm_reboot_notifier_all(vmid, 0, NULL);
580
581     return 0;
582 }

```

```

570 static int acrn_dev_release(struct inode *inode, struct file *filp)
571 {
572     struct acrn_vm *vm = filp->private_data;
573     int vmid = vm->vmid;
574
575     acrn_vm_destroy(vm);
576     kfree(vm);
577
578     // GRT:TODO add power service later.
579     guest_vm_reboot_notifier_all(vmid, 0, NULL);
580
581     return 0;
582 }

```

解决方案 (vmid=-1) -详情见谦川解析

• vmid =-1解决方案

- 将vm->saved_vmid 初始化移动到acrn_vm_create() 函数。保证vm结构体整个生命周期，vm->saved_vmid 一直是有效值。

```

82 > spin_lock_init(&vm->ioreq_clients_lock);
83 > vm->vmid = vm_param->vmid;
84 > vm->vcpu_num = vm_param->vcpu_num;
85
86 > if (acrn_ioreq_init(vm, vm_param->ioreq_buf) < 0) {
87 >     hcall_destroy_vm(vm_param->vmid);
88 >     vm->vmid = ACRN_INVALID_VMID;
89 >     return NULL;
90 > }
91
92 > write_lock_bh(&acrn_vm_list_lock);
93 > list_add(&vm->list, &acrn_vm_list);

```

```

149 >     vm->monitor_page = NULL;
150 > }
151
152 > vm->is_destroying = true;
153 > wake_up_interruptible(&vm->async_ioreqs_wq);
154 > acrn_async_ioreq_deinit(vm);
155
156 > acrn_vm_all_ram_unmap(vm);
157
158 > dev_dbg(acrn_dev.this_device, "VM %u destroyed.\n", vm->vmid);
159 > vm->saved_vmid = vm->vmid;
160 > vm->vmid = ACRN_INVALID_VMID;

```

```

82 > spin_lock_init(&vm->ioreq_clients_lock);
83 > vm->vmid = vm->saved_vmid = vm_param->vmid;
84 > vm->vcpu_num = vm_param->vcpu_num;
85
86 > if (acrn_ioreq_init(vm, vm_param->ioreq_buf) < 0) {
87 >     hcall_destroy_vm(vm_param->vmid);
88 >     vm->vmid = ACRN_INVALID_VMID;
89 >     return NULL;
90 > }
91
92 > write_lock_bh(&acrn_vm_list_lock);
93 > list_add(&vm->list, &acrn_vm_list);

```

acrn_vm_create() 函数

+55 common lines

+10

+10

```

149 >     vm->monitor_page = NULL;
150 > }
151
152 > vm->is_destroying = true;
153 > wake_up_interruptible(&vm->async_ioreqs_wq);
154 > acrn_async_ioreq_deinit(vm);
155
156 > acrn_vm_all_ram_unmap(vm);
157
158 > dev_dbg(acrn_dev.this_device, "VM %u destroyed.\n", vm->vmid);
159 > vm->vmid = ACRN_INVALID_VMID;

```

acrn_vm_destroy() 函数

MEDIATEK